

Mobile Programming Teoria

1 Fondamenti di Android

Cos'è Android

Android è uno stack software open-source basato su un kernel Linux. Non si tratta solo di un sistema operativo, ma di un ecosistema che include uno strato middleware, applicazioni native e un set completo di API e librerie. Il vero punto di forza della piattaforma è che **le applicazioni native e quelle create da sviluppatori di terze parti operano nel medesimo ambiente di runtime e condividono le stesse API**. Questo abbatte l'isolamento tra i software e garantisce a tutte le app un **accesso diretto e paritario alle funzionalità hardware** del dispositivo, favorendo una forte interazione e condivisione dei dati.

App Native e di Terze Parti

Applicazioni Native: Sono le app predefinite di sistema, progettate per gestire le **azioni comuni di base** (come l'app fotocamera o la posta elettronica). **Applicazioni di Terze Parti:** Oltre a introdurre **funzionalità del tutto nuove**, queste app possono registrarsi nel sistema per **sostituirsi alle app native**, offrendosi all'utente come valide alternative per eseguire le stesse operazioni di default.

L'approccio al mercato: Google vs Apple

Mentre l'ecosistema Apple (iPhone e iOS) è **chiuso e interamente proprietario**, l'approccio di Google con Android si basa su uno **stack software open source**. Supportato dalla Open Handset Alliance, Android si distingue per la sua **flessibilità**, essendo concepito per operare su **qualsiasi dispositivo mobile** che sia compatibile con i suoi requisiti hardware.

Il background di Android

Sebbene lo sviluppo per Android si basi sulla sintassi e sui principi fondamentali Object-Oriented del linguaggio Java (come l'uso di classi, interfacce e polimorfismo), **Android non è un'implementazione di Java ME**. Nel Java tradizionale, il codice sorgente diventa *bytecode* che viene poi interpretato da una Java Virtual Machine (JVM) standard, solitamente aiutata da un compilatore JIT (Just-In-Time) per ottimizzare le prestazioni in tempo reale. Android, invece, utilizza un **ambiente di esecuzione proprietario e ottimizzato per le limitate risorse dei dispositivi mobili** (storicamente la Dalvik Virtual Machine, oggi sostituita da ART - Android Run Time). A causa di questa differenza nell'ambiente di esecuzione, **le normali classi e i classici file eseguibili Java standard non possono funzionare nativamente sulla piattaforma Android**.

L'evoluzione dell'ambiente di esecuzione: da Dalvik ad ART

L'ambiente di esecuzione di Android non è sempre stato lo stesso:

- **Dalvik (JIT):** Utilizzata nelle versioni meno recenti di Android, questa macchina virtuale adottava la tecnologia Just-In-Time, compilando il codice dell'app in linguaggio macchina "in tempo reale" a ogni singola esecuzione. Pur essendo versatile, questo approccio causava un impatto negativo sulle prestazioni generali e sul consumo della batteria.
- **ART (AOT):** Il moderno Android Run Time ha sostituito Dalvik passando alla tecnologia Ahead-Of-Time. A differenza del JIT, ART compila interamente il codice in linguaggio macchina una sola volta, ovvero **durante l'installazione** dell'app.
- **Vantaggi e Compromessi:** L'approccio di ART garantisce prestazioni molto più fluide e un'ottimizzazione nettamente superiore delle risorse del sistema. Il lieve svantaggio (tempi di installazione leggermente superiori e maggiore spazio occupato in memoria) risulta impercettibile grazie alla potenza dei dispositivi odierni.

L'Architettura di Sistema di Android

Lo stack di Android è suddiviso in tre livelli principali:

- **Kernel Linux:** Gestisce le interazioni a basso livello con l'hardware, occupandosi di memoria, driver e processi.
- **Librerie Open Source:** Forniscono funzionalità essenziali al sistema e alle app, come database (SQLite), rendering web (WebKit), grafica (OpenGL) e multimedia (Media Manager).
- **Ambiente di Runtime (ART):** È l'ambiente in cui vengono eseguite le app, comprensivo delle librerie core di sistema.

Il Processo di Compilazione con Gradle

Durante la compilazione, il codice sorgente Java o Kotlin viene dapprima tradotto in bytecode Java (`.class`) e successivamente convertito dagli strumenti della build Android in file **DEX** (*Dalvik Executable*), il formato eseguibile utilizzato dalla piattaforma Android.

Struttura di un Progetto Android (File e Cartelle)

Il progetto si organizza in componenti specifici:

- **Configurazione (Gradle):** Il file `build.gradle` definisce configurazioni e dipendenze esterne, mentre `settings.gradle` elenca i moduli del progetto.
- **Descrittore (Manifest):** Il file `AndroidManifest.xml` descrive l'app al sistema, dichiarandone i componenti (come le Activity) e i permessi necessari.
- **Codice e Risorse:** `java/` : Ospita il codice sorgente (Java/Kotlin). `assets/` : Contiene file arbitrari accessibili all'app. `res/` : Raccoglie le risorse statiche, divise in sottocartelle specifiche: `layout/` per le interfacce, `drawable/` e `mipmap/` per grafica e icone, `values/` per stringhe, stili e dimensioni, `menu/` per i menu, `anim/` per le animazioni, `raw/` per dati grezzi e `xml/` per configurazioni generali

Sviluppo in Android Studio e Gestione dell'Interfaccia Grafica

Quando si crea un nuovo progetto in Android Studio, all'interno del package della cartella `java` viene generata automaticamente una classe iniziale:

- **La classe MainActivity:** Nei nuovi progetti, Android Studio genera la classe `MainActivity` (estensione di `AppCompatActivity`) per gestire i componenti visivi e interattivi.
- **View:** Tutti i componenti visuali dell'interfaccia utente (come pulsanti, campi di testo, etichette).
- **Disaccoppiamento tra Logica e Interfaccia:** C'è una netta separazione tra logica (scritta nel codice sorgente) e grafica. Quest'ultima viene definita esternamente in file **XML** all'interno della cartella `layout`.
- **Accesso alle Risorse tramite ID:** Per controllare l'interfaccia dal codice, si usa il metodo `setContentView()`, assegnando nei file XML degli **identificatori univoci (ID)** alle singole View in modo da poterle manipolare facilmente via codice.

L'Evoluzione dello Sviluppo: Android Jetpack

Con l'introduzione di Android 10, le precedenti Support Library si sono evolute in **Android Jetpack**, una suite completa di strumenti e linee guida creata per semplificare e velocizzare la realizzazione delle app. Il grande vantaggio di questo nuovo ecosistema è la sua natura modulare: i suoi componenti sono infatti forniti come librerie separate e indipendenti dalla piattaforma Android, consentendo così agli sviluppatori di integrare nei propri progetti solamente i moduli di cui hanno un reale bisogno, senza l'obbligo di importare l'intero pacchetto.

Componenti e Strumenti Fondamentali di Android

Android mette a disposizione degli sviluppatori una serie di componenti e strumenti che consentono di realizzare applicazioni modulari, interattive e integrate con il sistema operativo. I principali componenti applicativi devono essere dichiarati all'interno del file `AndroidManifest.xml`.

- **Activity:** rappresentano le singole schermate dell'applicazione con cui l'utente interagisce direttamente tramite l'interfaccia grafica. Sono implementate estendendo la classe `android.app.Activity` e vengono dichiarate nel Manifest tramite il tag `<activity>`.
- **Service (Servizi):** componenti privi di interfaccia grafica che eseguono operazioni in *background*, anche per periodi prolungati (riprodurre musica, scaricare file o sincronizzare dati). Sono implementati estendendo la classe `android.app.Service` e vengono dichiarati nel Manifest tramite il tag `<service>`.
- **Content Provider:** consentono di gestire e condividere in modo sicuro i dati dell'applicazione con altre applicazioni, astruendo i dettagli dell'implementazione del database. Vengono dichiarati nel Manifest tramite il tag `<provider>`.
- **Broadcast Receiver:** componenti in ascolto che reagiscono a eventi o messaggi (*broadcast*) inviati dal sistema operativo o da altre applicazioni. Vengono dichiarati nel Manifest tramite il tag `<receiver>`.

A supporto di tali componenti, Android fornisce ulteriori strumenti e meccanismi:

- **Intent:** oggetti utilizzati per avviare Activity e Service, comunicare tra componenti e notificare eventi tramite Broadcast Receiver.
- **Fragment:** porzioni modulari e riutilizzabili dell'interfaccia utente che possono essere integrate all'interno di una Activity. Possiedono un proprio ciclo di vita e consentono di realizzare interfacce più flessibili e dinamiche.
- **Widget:** componenti interattivi che possono essere collocati direttamente nella schermata Home del dispositivo per mostrare informazioni o fornire funzionalità rapide senza aprire l'applicazione.

- **Notifiche:** strumenti che permettono di informare l'utente del verificarsi di determinati eventi anche quando l'applicazione non è in primo piano.

Android offre inoltre diversi meccanismi di **persistenza dei dati**, che consentono alle informazioni generate dall'applicazione di essere conservate anche dopo la sua chiusura e rese disponibili nei successivi avvii.

I Vincoli Hardware e la Gestione delle Risorse

I dispositivi mobili presentano sfide specifiche:

- Processore e capacità di calcolo limitate.
- Quantitativo di memoria RAM ridotto.
- Spazio di archiviazione (storage) limitato.
- Batteria con un ciclo di vita limitato.

Il codice deve essere sviluppato in modo efficiente per garantire velocità e reattività, ottimizzando l'uso della memoria (evitando di creare oggetti inutili) e dando priorità all'autonomia della batteria.

Gestione dello Storage Sebbene le memorie siano migliorate, lo spazio va gestito con cura. Android offre diverse soluzioni in base alla mole di dati:

- **Grandi quantità di dati:** Sfruttare il database nativo **SQLite** e i **Content Providers** per salvare e condividere le informazioni.
- **Piccoli dati/Impostazioni:** Utilizzare le **Preferences** (SharedPreferences).
- **File generici:** Utilizzare il normale file system di sistema tramite la libreria `java.io`.

Sicurezza e Permessi

Permessi espliciti e responsabilità: L'app deve richiedere il permesso per accedere a funzionalità sensibili (come la fotocamera o i contatti). L'utente ha la responsabilità di decidere se concederli o negarli durante l'uso o l'installazione.

Qualsiasi tipo di permesso deve essere prima di tutto dichiarato nel file di configurazione principale dell'app. Questo passaggio è obbligatorio sia per i permessi normali (come l'accesso a Internet) sia per quelli più sensibili (come la geolocalizzazione).

Prevenzione (Hijacking): Oltre ai permessi, lo sviluppatore ha il compito di scrivere codice che impedisca che l'app venga "dirottata" (*hijacked*) da software malevoli.

Reattività e il Blocco delle App (ANR)

Android esige interfacce sempre reattive: se il thread principale si blocca per sovraccarico, il sistema interviene mostrando un errore **ANR** (Application Not Responding o *Force Close*) e chiede all'utente se forzare la chiusura dell'app.

- Il blocco viene innescato se l'app non risponde a un input entro **5 secondi** o se un componente *Broadcast Receiver* impiega più di **10 secondi** per completare il suo metodo `onReceive`.
- **La soluzione:** Spostare tutte le operazioni lunghe e complesse su **Thread in background** o **Service** per non bloccare mai l'interfaccia grafica.

Comportamento delle Applicazioni e Gestione del Ciclo di Vita

Nello sviluppo Android, le applicazioni possono essere classificate in base al modo in cui interagiscono con l'utente e alla loro modalità di esecuzione. Tale classificazione è strettamente legata alla gestione del ciclo di vita dell'applicazione e all'utilizzo delle risorse di sistema.

- **Foreground Activity:** sono applicazioni utilizzabili solo quando risultano visibili all'utente. Quando vengono coperte da un'altra schermata o minimizzate, passano in background e possono essere sospese o terminate dal sistema operativo per liberare memoria e risorse. Poiché il passaggio tra foreground e background avviene continuamente, è fondamentale che lo sviluppatore salvi lo stato dell'applicazione, ad esempio tramite la callback `onSaveInstanceState()` (un metodo del ciclo di vita usato per **salvare lo stato temporaneo dell'interfaccia utente** prima che un'Activity venga distrutta, evitando così la perdita dei dati inseriti).
- Le **Background Activity** operano in modo quasi totalmente autonomo e invisibile per l'utente, il quale interviene unicamente durante la configurazione iniziale del servizio. Queste applicazioni rimangono costantemente attive o reattive in background per gestire funzioni di sistema protettive o di monitoraggio continuo, come filtri antispam, antivirus o gestori di chiamate, elaborando i dati in tempo reale senza mai richiedere un avvio esplicito o mostrare un'interfaccia grafica durante il loro normale funzionamento.

- Le **Intermittent Activity** rappresentano una soluzione ibrida in cui l'utente avvia l'applicazione o ne abilita la funzione, ma la gestione dei singoli risvegli in background è interamente automatizzata. L'applicazione prosegue il suo lavoro principale anche quando l'interfaccia viene nascosta (come nel caso dei lettori musicali), alternando momenti di inattività a risvegli temporanei pianificati e regolati dal sistema operativo per scaricare dati o aggiornare i feed. In questo modo, l'app ottimizza l'uso della batteria pur continuando a comunicare il proprio stato all'utente attraverso notifiche o comandi rapidi di sistema.

Per garantire una corretta esperienza d'uso e rispettare le risorse del dispositivo, Android impone alcune linee guida fondamentali. Un'applicazione non dovrebbe continuare a consumare CPU, rete o batteria quando non è in primo piano; di conseguenza, gli aggiornamenti grafici e le operazioni non essenziali dovrebbero essere sospesi quando l'app viene minimizzata. Qualora sia necessario continuare a svolgere attività in background, queste devono essere delegate a componenti specifici come i **Service**, progettati per eseguire operazioni prolungate senza il peso dell'interfaccia grafica.

Anche la comunicazione con l'utente deve avvenire in modo non invasivo. Quando l'applicazione si trova in background, è opportuno utilizzare strumenti come le **Notifiche** o i messaggi **Toast**, evitando di interrompere bruscamente l'attività corrente dell'utente.

Un ulteriore requisito fondamentale riguarda la reattività dell'interfaccia. Android monitora costantemente il thread principale dell'applicazione e, se questo rimane bloccato troppo a lungo, genera un errore **ANR** (*Application Not Responding*). Per evitare questi problemi, tutte le operazioni lunghe o computazionalmente intensive devono essere eseguite su thread secondari o all'interno di Service, lasciando il thread principale libero di gestire l'interfaccia utente.

L'Activity: Il Cuore dell'Interfaccia Utente

L'Activity rappresenta una singola schermata che l'applicazione offre all'utente. Contiene almeno un layout e gestisce le interazioni con esso.

- **Creazione:** Per creare una nuova Activity è necessario estendere la classe base `android.app.Activity`.
- **Collegamento alla UI:** Appena creata deve essere collegata alla propria interfaccia (Layout, View o Fragment). Questo si ottiene invocando il metodo `setContentView()` all'interno della funzione di callback `onCreate()`.
- **Dichiarazione:** Per poter essere utilizzata, deve essere obbligatoriamente dichiarata nel file `AndroidManifest.xml` tramite il tag `<activity>`. All'interno di questo tag è possibile aggiungere un *Intent Filter* per specificare quali Intent possono avviarla.

Lo Stack delle Activity e i Suoi Stati

Lo stato di un'Activity e la sua priorità nel sistema sono determinati dalla posizione che occupa nel cosiddetto **Activity Stack**, una pila di tipo LIFO (Last-In, First-Out) che colleziona tutte le attività attualmente in esecuzione. Quando una nuova Activity viene avviata, si posiziona in cima allo stack e diventa visibile in primo piano. Se l'utente preme il tasto "Back" o l'Activity viene chiusa, essa viene rimossa dallo stack e l'Activity immediatamente sottostante torna in cima, diventando quella attiva.

| Durante il suo ciclo di vita, un'Activity entra ed esce da questo stack transitando attraverso quattro stati principali:

- **Active (Attiva):** L'Activity è in cima alla pila, visibile e in uso dall'utente. Il sistema la protegge fortemente dalla chiusura.
- **Paused (In Pausa):** Visibile solo parzialmente (ha perso il focus). Il sistema la termina solo in casi di estrema necessità.
- **Stopped (Fermata):** Completamente nascosta da altre schermate. Mantiene il suo stato in memoria, ma è la prima candidata a essere terminata se il sistema necessita di liberare risorse.
- **Inactive (Inattiva):** L'Activity è stata chiusa definitivamente e rimossa dalla pila. Per riutilizzarla, dovrà essere riaperta.

L'Android Memory Manager

Il passaggio tra gli stati di un'Activity (inclusa la terminazione per liberare memoria) è un processo non deterministico, gestito in totale autonomia dall'**Android Memory Manager**. In caso di RAM insufficiente, il sistema dà priorità alla chiusura delle app **Inattive** (Inactive), seguite da quelle **Fermate** (Stopped) e intacca quelle **In Pausa** (Paused) solo in casi di estrema necessità. Affinché l'esperienza utente sia fluida e senza interruzioni, questi passaggi di stato devono risultare invisibili a chi usa il dispositivo.

Il Ciclo di Vita (Lifetimes) e gli Eventi

Il ciclo di vita di un'Activity è scandito da specifiche funzioni di callback (gestori di eventi) e si articola in tre sotto-cicli principali.

Full Lifetime (Ciclo di vita completo). Inizia con `onCreate()` e termina con `onDestroy()`. Rappresenta l'intera esistenza di un'Activity, dalla sua creazione iniziale fino alla sua distruzione definitiva.

- `onCreate()` : Segna la creazione dell'Activity. Si costruisce l'interfaccia utente (es. tramite `setContentView()`), si allocano gli oggetti e si ripristina la UI tramite l'oggetto `Bundle` ricevuto in input. Per questioni di efficienza, bisogna **evitare di allocare troppi oggetti a vita breve** per non sovraccaricare inutilmente il *Garbage Collector*.
- `onDestroy()` : Ultima chiamata prima della distruzione. Si usa per liberare le risorse allocate in `onCreate()` e chiudere eventuali connessioni aperte (es. database o rete).

Visible Lifetime (Ciclo di vita visibile). Inizia con `onStart()` e termina con `onStop()`. L'Activity è visibile sullo schermo dell'utente. Tuttavia, **non è detto che sia in primo piano (foreground)** o che l'utente possa interagirci.

- `onStart()` / `onStop()` : Vengono chiamati quando l'app diventa visibile o viene nascosta. `onStop()` si usa per fermare animazioni, timer o thread legati unicamente all'aggiornamento visivo della UI.
- `onRestart()` : Viene invocato prima di un nuovo `onStart()` se l'Activity sta tornando visibile dopo essere stata precedentemente fermata.

Active Lifetime (Ciclo di vita attivo) Inizia con `onResume()` e termina con `onPause()`. L'Activity è completamente in **primo piano (foreground)** sullo schermo. In questo stato, l'utente interagisce direttamente con l'app (è l'Activity "in esecuzione" attuale).

- `onResume()` / `onPause()` : Un'Activity può passare molte volte tra questi due stati (es. quando lo schermo si spegne o arriva una chiamata).
- `onSaveInstanceState()` : Viene chiamato immediatamente prima di `onPause()`. È un passaggio fondamentale perché salva lo stato temporaneo della UI in un oggetto `Bundle`, permettendo poi di ripristinare la schermata esattamente com'era stata lasciata.

I Fragment: Modularità e Flessibilità

I **Fragment** (introdotti da Android 3.0) sono moduli UI riutilizzabili con proprio layout e ciclo di vita, nati per ridurre la complessità delle Activity. Permettono di adattare dinamicamente l'interfaccia a diversi schermi (es. smartphone vs tablet), migliorando notevolmente l'adattabilità della UX, anche se non sono strettamente obbligatori.

Dettagli Tecnici Fondamentali:

- A differenza delle Activity, i Fragment **non devono essere registrati** nel file `AndroidManifest.xml`, poiché non possono esistere autonomamente, ma devono sempre essere legati a un'Activity genitore.
- A partire da Android 10, la classe nativa `android.app.Fragment` è stata deprecata. Lo sviluppo moderno impone l'utilizzo dei Fragment forniti dalla **Support Library (AndroidX)**, garantendo così la retrocompatibilità su tutte le versioni.

Creazione e Aggiunta di un Fragment

Per creare un nuovo Fragment, è necessario estendere la classe base `Fragment` e definirne l'interfaccia utente e la logica.

(Nota: esistono anche Fragment privi di UI, noti come "Headless Fragment", utilizzati esclusivamente per eseguire operazioni in background).

L'aggiunta di un Fragment a un'Activity può avvenire in due modi:

Aggiunta Statica: Dichiarato direttamente nel file XML dell'Activity genitore. *Quando usarla:* È l'approccio ideale se la struttura della schermata è predefinita e non cambia durante l'esecuzione dell'app.

Aggiunta Dinamica: Aggiunto a *run-time* (tramite codice Java/Kotlin) in una `View` contenitore. Permette di modificare il Fragment mentre l'Activity è in esecuzione. *Quando usarla:* È l'approccio raccomandato se il layout deve cambiare in risposta ad azioni dell'utente o allo stato dell'applicazione.

Il Ciclo di Vita del Fragment

Il ciclo di vita di un Fragment è strettamente legato a quello dell'Activity che lo ospita, ma possiede metodi aggiuntivi per gestirne l'inserimento e la rimozione dinamica.

- **Creazione:** Inizia con l'associazione all'Activity (`onAttach()`), prosegue con la creazione logica (`onCreate()`) e il disegno dell'interfaccia (`onCreateView()`), per poi completarsi diventando visibile e interattivo (`onActivityCreated()`, `onStart()`, `onResume()`).

- **Pausa/Esecuzione:** Si sincronizza perfettamente con i passaggi di stato dell'Activity genitore (`onPause` , `onStop`).
- **Distruzione:** Prevede prima la rimozione della veste grafica (`onDestroyView`), poi la distruzione logica (`onDestroy`) e infine la disconnessione definitiva dall'Activity (`onDetach`).

Comunicazione e Gestione (FragmentManager)

Il coordinamento dei Fragment è affidato al **FragmentManager**, un componente che gestisce tutti i Fragment associati all'Activity corrente.

Ottenere i riferimenti: Un Fragment accede all'Activity tramite il metodo `getActivity()` ; viceversa, l'Activity recupera un Fragment specifico tramite il suo ID (es. `findFragmentById`) appoggiandosi al **FragmentManager**.

Comunicazione tra Fragment e Activity: Per segnalare eventi all'Activity genitore mantenendo il codice disaccoppiato, si deve definire un'interfaccia di **callback** all'interno del Fragment, obbligando poi l'Activity ospitante a implementarla.

Fragment Transactions (Transazioni)

L'aggiunta, rimozione o sostituzione di un Fragment a *run-time* avviene tramite le **Transaction**. Ogni set di modifiche applicato ai Fragment è considerato una singola transazione.

Si articola in passi precisi: acquisire il `FragmentManager` , avviare una `FragmentTransaction` , definire l'azione (`add` , `remove` , `replace`), eventualmente salvare nello storico (`addToBackStack`) e infine invocare obbligatoriamente il metodo `commit()` per applicare le modifiche.

Il Back Stack (Storico e Navigazione):

- **Senza `addToBackStack()`:** Il Fragment sostituito viene distrutto permanentemente, impedendo all'utente di ripristinarlo con il tasto "Back".
- **Con `addToBackStack()`:** Il Fragment non viene distrutto ma solo "stoppato". La transazione viene salvata nello storico, consentendo di tornare indietro con il tasto "Back" (un'azione che può essere simulata anche da codice chiamando `popBackStack()`).

2 Intent, Service e Multithreading

Gli Intent e la "Sandbox" di Android

Di base, le app mobili vengono eseguite in un ambiente isolato e sicuro chiamato **"sandbox"**, con accesso limitato al sistema. Per superare questi confini, Android utilizza gli **Intent**, un potente meccanismo di scambio messaggi (*message-passing*) che connette i componenti della stessa app o di app diverse, trasformando il dispositivo in un ecosistema interconnesso.

I Tre Usi Principali degli Intent

Gli Intent sono il motore che fa comunicare i componenti (*Activity*, *Service* e *Broadcast Receiver*). Vengono utilizzati principalmente per tre scopi:

1. **Avviare Componenti (Activity/Service):** Per dichiarare l'intenzione di avviare un'attività o un servizio, passandogli eventualmente una specifica porzione di dati su cui operare.
2. **Inviare Messaggi di Broadcast:** Per trasmettere notifiche a tutto il sistema o alle app in ascolto quando si verifica un evento (es. batteria scarica), supportando la creazione di architetture *event-driven*.
3. **Interagire con App Native:** Consentono il **disaccoppiamento dei componenti**. Invece di creare un'interfaccia da zero (es. per inviare un SMS o fare una chiamata), la nostra app può lanciare un Intent specifico e delegare il compito all'app nativa del sistema in ascolto per quell'azione.

Tipologie di Intent: Espliciti e Impliciti

Per avviare nuove attività, è possibile utilizzare due approcci distinti:

- **Intent Espliciti:** Si utilizzano quando si conosce esattamente il componente da avviare, indicando esplicitamente il contesto dell'app e la classe esatta dell'Activity. È l'approccio standard per la **navigazione interna** all'applicazione (passare da una schermata all'altra dell'app).
- **Intent Impliciti:** Dichiarano solo l'**azione da eseguire** (es. aprire una pagina web, scattare una foto) senza specificare quale applicazione, lasciando che sia il sistema operativo a trovare l'app più adatta.

Gestione dell'Avvio e Chiusura delle Activity

Per avviare le transizioni tra le schermate e farle comunicare, si usano metodi specifici dell'Activity.

- **Avvio:** Si utilizza **startActivity(intent)** per lanciare una nuova Activity, che viene posta in cima allo stack e resa visibile. Se si necessita di un *feedback* o di un risultato dalla nuova schermata, si utilizza invece **startActivityForResult(intent, requestCode)**.
- **Chiusura:** Un'Activity viene distrutta e rimossa dallo stack invocando il metodo **finish()** o tramite la pressione del tasto **"Back"** del dispositivo, riportando l'utente all'Activity precedente.

Gli Intent Impliciti e l'Intent Resolution

Quando si utilizza `startActivity()` con un **Intent implicito**, non viene indicata una specifica Activity da avviare. Il sistema operativo deve quindi individuare autonomamente il componente più adatto analizzando gli Intent Filter dichiarati dalle applicazioni installate, avviando un processo di selezione chiamato **Intent Resolution**. Durante questa fase, Android confronta le caratteristiche dell'Intent con i filtri disponibili: se esiste una sola Activity compatibile la avvia direttamente, mentre se più applicazioni possono gestire la stessa richiesta, mostra all'utente una finestra di scelta per selezionare quale utilizzare.

- **Costruzione:** Quando si crea un Intent implicito da passare al metodo `startActivity()`, invece di specificare il destinatario, si indicano l'**Azione** generale da compiere, i **Dati (URI)** su cui operare e gli eventuali **Dati Aggiuntivi (Extras)** necessari per fornire contesto.
- **Risoluzione (Intent Resolution):** A *run-time*, Android analizza l'Intent e cerca nel dispositivo l'app più idonea a gestirlo; se esistono più applicazioni compatibili, mostra una finestra di dialogo per far scegliere l'utente.
- **Vantaggi:** Permettono di sviluppare app più leggere e modulari riutilizzando le funzionalità di app già installate (ad esempio, delegando una chiamata all'app "Telefono" nativa passando semplicemente il numero come URI).

Comunicazione Bidirezionale: Il concetto di Sub-Activity

Quando un'Activity viene avviata tramite `startActivity()`, essa opera in modo indipendente e non restituisce informazioni all'Activity che l'ha aperta. In alcune situazioni, però, è necessario ottenere un risultato da una schermata secondaria, ad esempio quando l'utente deve selezionare una foto o un contatto. In questi casi si utilizza una **Sub-Activity**, che non rappresenta una tipologia speciale di Activity, ma una normale Activity avviata con lo scopo di restituire dati al termine della sua esecuzione.

Per avviare un'Activity in attesa di un risultato si utilizza `startActivityForResult()`, passando sia l'Intent sia un **Request Code**, un identificatore numerico che consente di distinguere le diverse richieste e riconoscere quale operazione ha generato il risultato ricevuto.

Al termine dell'elaborazione, la Sub-Activity restituisce il risultato tramite `setResult()` prima di eseguire `finish()`. Attraverso questo metodo può comunicare l'esito dell'operazione (`RESULT_OK`, `RESULT_CANCELED` o valori personalizzati) ed eventuali dati da restituire all'Activity chiamante. Se l'utente interrompe l'operazione con il tasto Back o l'Activity termina senza chiamare `setResult()`, il sistema restituisce automaticamente `RESULT_CANCELED` e un Intent nullo.

Questo meccanismo consente una vera **comunicazione bidirezionale** tra Activity. Quando la Sub-Activity termina, infatti, l'Activity principale riceve una notifica tramite il callback `onActivityResult()`, che fornisce il **requestCode** associato alla richiesta, l'esito dell'operazione e i dati eventualmente restituiti. In questo modo l'Activity chiamante può verificare il risultato ottenuto e aggiornare il proprio comportamento di conseguenza.

💡 Nelle versioni più recenti di Android, `startActivityForResult()` e `onActivityResult()` sono stati deprecati e sostituiti dalla **Activity Result API** (`registerForActivityResult()`). Tuttavia, il principio di funzionamento rimane invariato: un'Activity avvia un'altra Activity, attende il completamento dell'operazione e riceve infine un risultato.

Le Azioni Native di Android (Activity Intents)

La classe `Intent` fornisce una serie di azioni native, definite come costanti stringa statiche. Quando si creano *Intent impliciti*, è possibile utilizzare queste costanti per richiedere al sistema l'avvio di Activity o Sub-Activity in grado di gestire l'operazione desiderata.

Di seguito un elenco raggruppate per categoria d'uso:

Telefonia e Comunicazione

- `ACTION_ANSWER` : Apre un'attività per gestire e rispondere alle chiamate in entrata.

- **ACTION_DIAL** vs **ACTION_CALL**: Entrambe gestiscono le telefonate. **DIAL** precompila il numero nel tastierino ma lascia l'avvio della chiamata all'utente (fortemente consigliato), mentre **CALL** avvia la chiamata immediatamente ma necessita di permessi di sicurezza elevati.
- **ACTION_SENDTO**: Permette di inviare un messaggio a un destinatario specifico indicato tramite l'URI (es. per SMS o email).

Condivisione e Trasferimento Dati

- **ACTION_SEND**: È l'azione standard per condividere contenuti verso un destinatario remoto. Per funzionare correttamente, richiede di definire il formato dei dati tramite `setType()` (MIME type) e di inserire i dati effettivi (testi, file multimediali o campi email) all'interno degli *Extras* dell'Intent (es. `EXTRA_TEXT`, `EXTRA_STREAM`).

Visualizzazione e Ricerca

- **ACTION_VIEW**: È l'azione più usata in assoluto per la visualizzazione. Il sistema sceglie in automatico l'app migliore in base al prefisso dell'URI fornito (es. `http`: apre il browser, `tel`: il tastierino, `geo`: Google Maps).
- **ACTION_SEARCH** e **ACTION_WEB_SEARCH**: **SEARCH** lancia una ricerca locale (il termine da cercare va passato negli extras tramite la chiave `SearchManager.QUERY`), mentre **WEB_SEARCH** apre un'Activity (come il browser) per cercare il testo direttamente sul web.

Gestione dei Dati (CRUD)

- **ACTION_PICK**: Avvia una Sub-Activity con interfaccia dinamica che fa selezionare un dato all'utente (es. un contatto in rubrica) e restituisce l'URI dell'elemento scelto all'app genitore.
- **ACTION_INSERT** / **ACTION_EDIT** / **ACTION_DELETE**: Servono rispettivamente ad aprire schermate per: inserire un nuovo elemento nel contenitore indicato (restituendo l'URI del nuovo dato), modificare un dato specifico, oppure eliminare l'elemento puntato dall'URI.

Cosa sono gli Intent Filter

Gli **Intent Filter** dichiarano a quali tipi di Intent un componente Android può rispondere. Vengono utilizzati nel file `AndroidManifest.xml` (tramite il tag `<intent-filter>`) per registrare **Activity**, **Service** e **Broadcast Receiver**, rendendoli disponibili le proprie funzionalità anche ad app terze. Permettono di intercettare gli **Intent impliciti** specificando tre elementi chiave:

- **Action**: L'azione generica da compiere (es. `ACTION_VIEW`).
- **Data**: Il tipo di dati accettati (es. un URI o un MIME type).
- **Category**: Il contesto del componente (es. `CATEGORY_LAUNCHER`).

I Tag Fondamentali di un Intent Filter

Le caratteristiche di un Intent Filter vengono definite attraverso tre tag principali:

- `<action>`: indica l'azione che il componente è in grado di eseguire. Ogni Intent Filter deve contenere almeno un tag di questo tipo.
- `<data>`: definisce i tipi di dati supportati dal componente, ad esempio specifici URI o formati di file.
- `<category>`: specifica il contesto o le condizioni in cui l'azione può essere eseguita. È possibile definire più categorie all'interno dello stesso filtro.

Il Tag `<data>` Per descrivere con precisione i dati gestibili, il tag `<data>` può utilizzare diversi attributi per individuare automaticamente il componente più adatto a gestire un determinato Intent. Tra questi:

- `android:scheme`: lo schema dell'URI (ad esempio `http`, `https`, `content`, `tel`);
- `android:host`: il nome dell'host associato all'URI;
- `android:port`: la porta utilizzata dall'host;
- `android:path`: il percorso specifico dell'URI;
- `android:mimeType`: il tipo di dato o formato supportato, come `image/jpeg` o `text/plain`.

Le Categorie Standard (`<category>`)

Android mette a disposizione diverse categorie standard che permettono di specificare il contesto in cui un componente può essere utilizzato. Tra le più importanti troviamo:

- **DEFAULT** : rende l'Activity disponibile per la gestione degli **Intent impliciti**. Senza questa categoria, il sistema non considererà l'Activity durante il processo di ricerca del componente più adatto.
- **BROWSABLE** : permette all'Activity di essere richiamata da un browser web, ad esempio per aprire direttamente nell'app determinati collegamenti Internet.
- **LAUNCHER** : fa sì che l'Activity compaia tra le applicazioni avviabili dal launcher del dispositivo.
- **HOME** : indica che l'app può svolgere il ruolo di schermata Home alternativa del sistema.
- **ALTERNATIVE** e **SELECTED_ALTERNATIVE** : consentono di definire azioni alternative rispetto a quella principale associata a un determinato tipo di dato.
- **GADGET** : specifica che l'Activity può essere incorporata all'interno di un'altra Activity.

Il Processo di Intent Resolution (Fase per Fase)

Il processo di Intent Resolution avviene in diverse fasi:

- **Raccolta dei candidati**: il sistema esamina tutti gli Intent Filter presenti sul dispositivo.
- **Verifica di azioni e categorie**: vengono mantenuti soltanto i filtri che supportano l'azione richiesta e che includono tutte le categorie specificate nell'Intent.
- **Verifica dei dati**: Android confronta eventuali URI o tipi MIME dell'Intent con le specifiche definite nel tag `<data>`. I filtri incompatibili vengono esclusi.
- **Scelta finale**: se rimane un solo componente compatibile, questo viene avviato automaticamente; se invece ne esistono più di uno, l'utente può scegliere quale applicazione utilizzare.

Questo meccanismo consente al sistema di individuare in modo dinamico il componente più adatto per eseguire una determinata operazione.

Priorità delle App

Nel processo di Intent Resolution, Android non attribuisce privilegi particolari alle applicazioni di sistema rispetto a quelle installate dall'utente. Tutti i componenti partecipano alla selezione seguendo le stesse regole e sulla base degli Intent Filter dichiarati.

Di conseguenza, un'applicazione di terze parti può sostituire il comportamento predefinito del sistema per determinate azioni, come la navigazione web, la gestione degli SMS o l'apertura di specifici tipi di file. Questo approccio garantisce una maggiore flessibilità e permette all'utente di scegliere liberamente quali applicazioni utilizzare come predefinite.

Gestione e Lettura dei Dati nell'Activity Destinataria

Quando un'Activity viene avviata tramite un Intent implicito, deve poter accedere alle informazioni contenute nell'Intent per capire quale operazione eseguire e su quali dati lavorare. Generalmente questa operazione viene effettuata all'interno del metodo `onCreate()`.

Per leggere le informazioni ricevute si utilizzano diversi metodi:

- `getIntent()` : restituisce l'Intent che ha avviato l'Activity.
- `getAction()` : permette di conoscere l'azione richiesta.
- `getData()` : recupera l'URI associato ai dati da elaborare.
- `getExtras()` o metodi come `getStringExtra()` e `getIntExtra()` : consentono di leggere eventuali parametri aggiuntivi allegati all'Intent.

In questo modo l'Activity può interpretare correttamente la richiesta ricevuta e adattare il proprio comportamento ai dati forniti.

Popolamento Dinamico dei Menù tramite Intent Filter

Android consente di arricchire dinamicamente i menù di un'applicazione sfruttando gli Intent Filter dichiarati dalle altre app installate sul dispositivo. Attraverso il metodo `addIntentOptions()`, lo sviluppatore può fornire un Intent che descrive un determinato tipo di dato o di azione: il sistema esegue quindi una ricerca tra tutti i filtri compatibili e genera automaticamente nuove voci di menù corrispondenti alle applicazioni in grado di gestire quella richiesta.

Questo meccanismo bidirezionale permette a qualsiasi applicazione di terze parti di rendere disponibili le proprie funzionalità ai menù di altre app. Per farlo, è necessario dichiarare nel file `AndroidManifest.xml` un `<intent-filter>` che specifichi le azioni supportate (tramite `<action>`), i tipi di dati gestibili (tramite `<data>`) e le categorie speciali `ALTERNATIVE` o `SELECTED_ALTERNATIVE`, indispensabili per l'integrazione dinamica. Infine, l'attributo `android:label` definito nel filtro verrà utilizzato dal sistema come etichetta testuale per la voce di menù mostrata all'utente.

Messaggi di Sistema: Intent Broadcast e Receivers

Oltre a essere utilizzati per avviare Activity e Service, gli **Intent** rappresentano il principale meccanismo di comunicazione asincrona di Android. Attraverso gli **Intent Broadcast** è possibile inviare messaggi a tutto il sistema per notificare il verificarsi di un determinato evento.

I broadcast seguono il paradigma della programmazione **event-driven** e consentono ai componenti di reagire a eventi generati da altre applicazioni o dal sistema operativo stesso. Android utilizza ampiamente questo meccanismo per segnalare eventi come il cambio di stato della batteria, la connessione a una rete Wi-Fi o la ricezione di una chiamata. Per inviare un messaggio broadcast è sufficiente creare un oggetto `Intent` e richiamare il metodo `sendBroadcast()`.

Comunicare senza UI: Toast e Notifiche

Quando un'app opera in background, ha la necessità di comunicare con l'utente senza rubare il *focus* o interrompere le sue azioni correnti. Android fornisce due tecniche:

- **I Toast:** Sono messaggi a comparsa brevi, silenziosi e temporanei. Appaiono in sovrapposizione per pochi secondi e svaniscono automaticamente, risultando ideali per feedback rapidi.
- **Le Notifiche (Notification):** Rappresentano il meccanismo principale per gli avvisi di maggiore importanza. Generano un'icona nella barra di stato del sistema e, aprendo la tendina, mostrano i dettagli dell'evento, consentendo spesso interazioni dirette (come rispondere a un messaggio).

Broadcast Receiver

Il componente incaricato di ricevere e gestire gli Intent Broadcast è il **Broadcast Receiver**. Per crearne uno, è necessario estendere la classe `BroadcastReceiver` e implementare il metodo `onReceive()`, che viene eseguito automaticamente quando arriva un messaggio compatibile con gli Intent Filter dichiarati.

Registrazione dei Broadcast Receiver

Affinché un Broadcast Receiver possa ricevere i messaggi, deve essere registrato tramite un Intent Filter. Esistono due modalità principali di registrazione.

- **Registrazione nel Manifest:** il Receiver viene dichiarato all'interno del file `AndroidManifest.xml`. Può ricevere broadcast anche quando l'applicazione non è in esecuzione. È particolarmente utile per intercettare eventi di sistema. Nelle versioni più recenti di Android il suo utilizzo è stato limitato per motivi di sicurezza e ottimizzazione energetica.
- **Registrazione via Codice:** il Receiver viene registrato dinamicamente durante l'esecuzione dell'applicazione. Rimane attivo solo finché il componente che lo ha registrato è in esecuzione. È la soluzione più adatta per aggiornare l'interfaccia utente in risposta a eventi temporanei. Generalmente viene registrato nei metodi `onStart()` o `onResume()` e rimosso in `onStop()` o `onPause()`.

Il Vincolo del Tempo: Deadline di onReceive

È cruciale prestare attenzione a come si scrive il codice all'interno del Receiver.

- Quando viene catturato un Intent Broadcast, il metodo `onReceive()` viene eseguito sul **Thread Principale** dell'applicazione.
- L'elaborazione all'interno di questa funzione non deve **mai superare i 10 secondi**. Se un'operazione lenta (es. calcoli pesanti o download di rete) sfiora questo limite temporale, il sistema operativo considererà l'app "non rispondente", genererà un errore **ANR** (*Application Not Responding*) e proporrà all'utente di chiuderla forzatamente.
- Per operazioni lunghe, il Receiver dovrebbe avviare un *Service* o un processo asincrono (come un *Job*).

I Service: Elaborazione in Background

A differenza delle Activity, i **Service** sono componenti privi di interfaccia grafica, progettati per gestire operazioni prolungate in modo invisibile.

- **Priorità di Sistema:** Godono di una priorità maggiore rispetto alle Activity in background, riducendo drasticamente il rischio di essere terminati dal Memory Manager.
- **Esecuzione Continua:** Permettono alle funzionalità "dietro le quinte" di continuare a operare anche se l'app viene minimizzata o chiusa.
- **Controllo dei Componenti:** Possono essere avviati o interrotti da altri componenti (Activity, Broadcast Receiver) e sono indispensabili per task slegati dall'input dell'utente (es. riprodurre musica o scaricare file).

- **Service di Sistema:** Android stesso si basa su questo paradigma, offrendo nativamente numerosi servizi come il *Location Manager* (GPS), il *Media Controller* o il *Notification Manager*.

Nota sui Foreground Service: È possibile innalzare la priorità di un Service per equipararla a quella di un'Activity in primo piano, ma in questo caso è obbligatorio mostrare una notifica persistente all'utente.

Implementare un Service: Ciclo di Vita

Per implementare un Service personalizzato bisogna creare una classe che estenda la classe base `Service` e registrarla obbligatoriamente nel file `AndroidManifest.xml`. È necessario sovrascrivere i seguenti metodi fondamentali:

- `onCreate()` : Viene invocato esclusivamente alla creazione iniziale del Service.
- `onBind` : È un metodo obbligatorio. Se il servizio non è di tipo "associato" (cioè non richiede un'interfaccia di comunicazione costante), deve semplicemente restituire `null`.
- `onStartCommand` : È il cuore operativo del componente; viene richiamato **ogni singola volta** che viene invocata la funzione `startService()`. *Valore di ritorno:* Questo metodo deve restituire un valore intero. Tale valore serve a indicare al sistema operativo come comportarsi in caso di chiusura forzata del Service per mancanza di memoria, stabilendo se e come il componente debba essere riavviato automaticamente appena le risorse tornano disponibili. I valori principali sono: `START_STICKY` : Se il sistema uccide il Service, lo ricrea non appena possibile. In questo caso, il metodo `onStartCommand()` viene nuovamente invocato passando un Intent `null`. È la scelta ideale per servizi che rimangono in esecuzione a tempo indeterminato e gestiscono lo stato in autonomia, come un riproduttore musicale. `START_NOT_STICKY` : Se il sistema uccide il Service, questo non viene ricreato automaticamente. Il componente rimarrà arrestato finché l'applicazione non eseguirà una nuova chiamata esplicita a `startService()`. Si utilizza per compiti specifici e limitati nel tempo, come il caricamento di un file su un server, dove un riavvio automatico senza dati aggiornati sarebbe inutile o dannoso.

Attenzione: Il Tranello del Main Thread

I Service, per impostazione predefinita, operano all'interno del **Main Thread** (o UI Thread) dell'app. Qualsiasi operazione lunga o bloccante inserita direttamente in `onStartCommand()` bloccherà l'interfaccia utente, portando al crash dell'applicazione con un errore **ANR** (*Application Not Responding*). **La Soluzione:** Per elaborare dati in background, lo sviluppatore deve esplicitamente avviare un **nuovo Thread** (o usare costrutti come le *Coroutines*) all'interno di `onStartCommand()`. A lavoro ultimato, il Service deve auto-terminarsi invocando il metodo `stopSelf()`.

I "Foreground Service": Priorità e Sicurezza

In situazioni in cui un Service esegue azioni importanti per l'utente (come la riproduzione musicale o la traccia del GPS), bisogna evitare che il sistema lo uccida per mancanza di risorse. Lo si fa promuovendolo a **Foreground Service** (Servizio in primo piano) con il metodo `startForeground()`.

Essendo un processo ad altissima priorità che impatta sulla batteria, Android impone la massima trasparenza verso l'utente. Pertanto, `startForeground()` richiede come parametro obbligatorio un oggetto **Notification**. Questa notifica sarà persistente (non chiudibile con un semplice *swipe*) e resterà fissa nella barra di stato per tutto il tempo in cui il servizio sarà in *foreground*.

Processi in Background e il Rischio ANR

Per mantenere l'interfaccia reattiva, operazioni come accessi al file system, chiamate di rete, transazioni su database e calcoli complessi **non devono mai** essere eseguite sul *Main Thread* vanno obbligatoriamente delegate a **processi in background** (**Thread secondari**). Storicamente, Android offre due approcci principali: l'utilizzo diretto dei **Thread Java** (con la classe `Handler`) e l'uso della classe **AsyncTask**. Ecco il testo revisionato. Ho eliminato la ridondanza finale sui Thread e gli Handler, unendola direttamente all'introduzione per creare un flusso logico unico e più scorrevole, formattando il tutto per una migliore leggibilità.

I Thread e la Classe Handler

In Android vige una regola rigorosa: solo il **Main Thread** (o UI Thread) può modificare l'interfaccia utente. Per eseguire operazioni pesanti in background mantenendo un controllo granulare sull'esecuzione, si istanzia la classe standard `java.lang.Thread` (Worker Thread). Tuttavia, se questo Thread secondario cerca di aggiornare direttamente un elemento grafico, l'app va in crash.

Per risolvere il problema di comunicazione si sfrutta `android.os.Handler`. Questa classe agisce da "ponte" sicuro: permette al thread in background di inviare messaggi alla coda del Main Thread, che si occuperà di elaborare la richiesta e aggiornare l'interfaccia grafica in totale sicurezza.

La Classe AsyncTask (Gestione Semplificata)

Nota tecnica: Classe formalmente deprecata da Android 11 per evitare *memory leaks* in favore di Coroutines o RxJava, ma fondamentale per comprendere il codice legacy.

In alternativa all'uso esplicito di Thread e Handler, `AsyncTask` forniva un meccanismo ad alto livello per eseguire task pesanti in background e aggiornare l'interfaccia utente in automatico. **Struttura Generica** Estendendo la classe, è necessario definire tre tipi di dati:

- **Params (Input):** I parametri passati al task per iniziare l'elaborazione.
- **Progress:** Il tipo di dato (es. un intero) per misurare lo stato di avanzamento.
- **Result (Risultato):** Il dato finale restituito al termine dell'operazione.

I Metodi Fondamentali

- `doInBackground(Params...)` : È l'unico metodo eseguito nel thread secondario. Contiene il codice pesante e al suo interno non si può **mai** interagire con la UI. Utilizza la funzione `publishProgress()` per segnalare l'avanzamento dell'operazione.
- `onProgressUpdate(Progress...)` : Eseguito sul Main Thread ogni volta che viene invocato `publishProgress()`. Serve ad aggiornare lo stato dell'interfaccia in tempo reale (es. una `ProgressBar`).
- `onPostExecute(Result)` : Eseguito sul Main Thread appena terminato il lavoro in background. Riceve il risultato finale per aggiornare definitivamente la UI.

Esecuzione Il processo si avvia istanziando la classe e richiamando il metodo `execute(Params)`. **Vincolo cruciale:** Ogni istanza è "usa e getta" e può essere eseguita una sola volta; per ripetere il task bisogna creare un nuovo oggetto.

3 Risorse, UI e Layout

L'Eternalizzazione delle Risorse in Android

Nello sviluppo Android, l'**esternalizzazione delle risorse** (come immagini, testi e layout) dal codice sorgente è considerata una pratica fondamentale. Separare le risorse facilita la loro manutenzione e permette al sistema di utilizzare automaticamente set di risorse alternative in base alla lingua o alle caratteristiche hardware del dispositivo (es. orientamento dello schermo).

Cartella res/: È la directory del progetto in cui vengono memorizzate e organizzate tutte le risorse, suddivise in sottocartelle specifiche per tipologia. **La Classe R:** Durante la compilazione, le risorse vengono ottimizzate e impacchettate; in questa fase il sistema autogenera la **classe R**, che contiene gli identificatori univoci necessari per richiamare facilmente e in modo sicuro ogni risorsa direttamente all'interno del codice.

Le Categorie Principali di Risorse

Esistono 7 categorie principali di risorse in Android (`values`, `drawables`, `layouts`, `animations`, `xml`, `styles` e `raw`). Di seguito un'analisi dettagliata delle più utilizzate:

- **Values (Valori Semplici):** Salvati come file XML in `res/values/` (es. `strings.xml`, `colors.xml`, `dimens.xml`).
Includono: *Strings*: Definite dal tag `<string>`, supportano formattazioni HTML di base come grassetto, corsivo e sottolineato. *Colors*: Definiti dal tag `<color>`, utilizzano notazioni esadecimali per i colori (incluso il canale Alpha per la trasparenza, es. `#ARGB`). *Dimensions*: Definite dal tag `<dimen>` per le costanti di layout. Impiegano unità fisiche (`px`, `in`, `pt`, `mm`) o unità raccomandate per adattarsi a diversi schermi: **dp** (per le interfacce) e **sp** (per i testi, tenendo conto delle preferenze utente).
- **Drawables (Elementi Grafici):** Cartella `res/drawable/` riservata a immagini (PNG, JPG, GIF) e formati ridimensionabili come i `NinePatch`. *Nota:* Da Android 4.3, le icone dell'app (launcher icons) vanno inserite nell'apposita cartella `mipmap/`.
- **Layouts:** File XML in `res/layout/` che disaccoppiano la logica dell'app dalla sua presentazione visiva; vengono "disegnati" nell'Activity tramite il metodo `setContentView()`.
- **Styles (Stili):** Consentono di mantenere un aspetto grafico coerente per le View definendo attributi tramite i tag `<style>` e `<item>`. Sfruttando l'attributo `parent`, supportano l'ereditarietà per estendere stili preesistenti.

Le Animazioni

Gestire le animazioni come risorse esterne permette di riutilizzarle in più schermate e di caricarne versioni differenti in base all'hardware. Si dividono in due tipologie:

- **Tweened Animations** (salvate in `res/anim/`): Servono a trasformare una singola View applicando rotazione, movimento, ridimensionamento o modifiche alla trasparenza. I loro principali attributi XML controllano il tempo totale (`duration`), il ritardo iniziale per eseguirle in sequenza (`startOffset`), il mantenimento dello stato prima e dopo l'esecuzione (`fillBefore` / `fillAfter`) e la curva di accelerazione nel tempo (`interpolator`).
- **Frame by Frame** (salvate in `res/drawable/`): Animazioni create mostrando in sequenza diverse immagini per un tempo prestabilito.

Risorse Alternative e Qualificatori

I qualificatori per le risorse Android sono suffissi (es. `-land`, `-en`, `-sw600dp`) aggiunti alle cartelle nel percorso `/res` del progetto. È il sistema che permette ad Android di scegliere automaticamente a *run-time* layout, immagini e testi specifici in base alle caratteristiche del dispositivo o alle preferenze dell'utente (lingua, rotazione, risoluzione). Si implementa creando cartelle parallele in `res/` e aggiungendo al nome dei **qualificatori** separati da un trattino (`-`). Se non viene trovata una cartella con i qualificatori che corrispondono alle condizioni del dispositivo, cercherà la corrispondenza più vicina, per poi ripiegare infine sulla cartella standard (nessun qualificatore).

- **Gerarchia e Regole:** L'ordine dei qualificatori è tassativo e non si possono inserire due qualificatori della stessa categoria nel nome della cartella.
- **Tipi di qualificatori:** In ordine di priorità, i principali sono: Lingua (es. `en`), Regione (es. `rus`), Orientamento dello schermo (`port` o `land`), Densità dei pixel, Tipo di touch screen, Disponibilità e Tipo di tastiera, e infine la Risoluzione esatta dello schermo.

I Componenti Base dell'Interfaccia (UI)

Per gestire l'interfaccia utente, Android introduce una struttura gerarchica:

- **Activity:** le singole finestre visibili all'utente, all'interno delle quali vengono posizionati e organizzati View e ViewGroup.
- **View:** Le classi base per la rappresentazione visiva di tutti gli elementi grafici (i "controlli" o widget). Qualsiasi elemento disegnato sullo schermo deriva da questa classe base.
- **ViewGroup:** Sono estensioni della classe View progettate per fare da **contenitori**. Ospitano al loro interno View "figlie" (o altri ViewGroup) e servono per creare controlli personalizzati o per gestire la disposizione grafica tramite i *Layout Manager*.

Sebbene Android fornisca nativamente un'ampia gamma di controlli e layout, nella maggior parte delle applicazioni reali gli sviluppatori estendono o modificano questi controlli standard per introdurre funzionalità personalizzate.

Principali Classi View (Controlli Standard)

Android mette a disposizione diversi widget pronti all'uso per l'interazione con l'utente:

- **TextView:** Etichetta di testo in sola lettura (*read-only*), con supporto per multilinea, formattazione di base e a capo automatico.
- **EditText:** Casella di testo modificabile dall'utente, che supporta l'inserimento multilinea e i testi di suggerimento temporanei (*hint*).
- **ListView:** Un ViewGroup che crea e gestisce una **lista scorrevole verticalmente**. Nella sua forma base, dedica una TextView a ogni riga mostrata.
- **Spinner:** Un menù a tendina (*drop-down*) che mostra il valore selezionato e, al tocco, apre una finestra con una lista per scegliere un nuovo elemento.
- **Button:** Il classico pulsante cliccabile.
- **CheckBox:** Pulsante a due stati che consente all'utente di spuntare o deselezionare una specifica opzione.

I Layout (Layout Managers)

I Layout sono particolari ViewGroup che servono a disporre e dimensionare le View figlie sullo schermo, con la possibilità di essere annidati per creare interfacce complesse. I principali layout nativi sono:

- **FrameLayout:** È il più semplice. Posiziona gli elementi nell'angolo in alto a sinistra, sovrapponendoli uno sull'altro come in un mazzo di carte.
- **LinearLayout:** Dispone le View in un'unica riga orizzontale o colonna verticale, permettendo di distribuire lo spazio in modo proporzionale assegnando un "peso" agli elementi.
- **RelativeLayout:** Permette di posizionare le View in base a regole relative: rispetto ad altre View (es. "sotto al pulsante X") o rispetto ai bordi del display (es. "allineato in basso a destra").
- **TableLayout:** Dispone le View all'interno di una griglia composta da righe e colonne.

- **Gallery:** Mostra elementi in una riga scorrevole orizzontale (oggi deprecato in favore di soluzioni moderne e più efficienti).

Definizione e Implementazione dei Layout

Esistono due approcci per implementare i Layout:

- **L'approccio XML (Preferito):** Si utilizza un file di risorse esterno XML caratterizzato da un singolo "nodo radice". È l'approccio preferito perché separa nettamente il design grafico dalla logica di programmazione.
- **L'approccio via Codice:** Sebbene sia meno pratico per la progettazione visiva, permette di istanziare e configurare dinamicamente Layout e View scrivendo direttamente il codice Java o Kotlin.

Regole per App Veloci: Ottimizzazione della UI

Poiché il processo di disegno dell'interfaccia (*rendering*) è molto dispendioso per il sistema, è cruciale progettare layout leggeri per non compromettere la reattività dell'app. Le best-practice impongono di: **Evitare annidamenti non necessari:** Bisogna sempre rimuovere i layout ridondanti (ad esempio layout che contengono un solo figlio) che spesso si creano durante le modifiche ripetute all'interfaccia. **Non usare troppe View:** Poiché ogni elemento aggiuntivo richiede tempo di calcolo, è buona regola non superare il limite di **80 View** complessive per singola schermata. **Evitare gerarchie troppo profonde:** Non bisogna mai superare i **10 livelli massimi di annidamento** (layout dentro layout).

Strumenti di Analisi: Android Lint

Lint è uno strumento ufficiale dell'SDK di Android che effettua un'analisi statica dei file di progetto (inclusi i file XML) restituendo avvisi e raccomandazioni per correggere errori strutturali e migliorare le performance. **Come si usa:** Oggi è comodamente integrato all'interno di Android Studio per evidenziare gli errori in tempo reale, ma può essere utilizzato anche da riga di comando con la sintassi `lint [flags] <directory_del_progetto>`, previa aggiunta del percorso dei *tools* dell'SDK alla variabile d'ambiente `PATH` del sistema.

Creazione di View Personalizzate (Custom Views)

Android offre un'enorme flessibilità, permettendo agli sviluppatori di creare nuove View tramite il *subclassing* (creazione di sottoclassi) di componenti esistenti o l'implementazione da zero di nuovi controlli. Gli approcci principali sono tre:

- **Modifica o Estensione di View esistenti:** Si personalizza un controllo standard esistente (ad esempio, sovrascrivendo il metodo `onDraw`) per alterarne l'aspetto o il comportamento senza doverne riscrivere le funzioni di base.
- **Controlli Composti (Compound Controls):** Si combinano diverse View standard all'interno di un `ViewGroup` (come un `Layout`) per formare un blocco unico e riutilizzabile, gestendone la logica via codice.
- **Creazione da Zero (From Scratch):** Si realizza un controllo del tutto nuovo quando l'interfaccia richiesta si discosta radicalmente da quanto offerto nativamente dal framework.

Il Compromesso tra Estetica e Usabilità

Nella progettazione dell'interfaccia, è essenziale bilanciare la libertà creativa con la reale usabilità. È raccomandabile resistere alla tentazione di sviluppare ogni elemento da zero ("tutto custom") e preferire l'uso di controlli standard: questi risultano immediatamente familiari agli utenti, riducendo la curva di apprendimento, e ricevono in automatico correzioni e aggiornamenti estetici a ogni nuova release di Android.

Sviluppare View Personalizzate ex novo (2D e 3D)

La creazione di controlli da zero garantisce il massimo controllo sulle modalità di interazione dell'app. Per creare un nuovo controllo è necessario estendere una delle due classi grafiche fondamentali:

- **Classe View (Grafica 2D):** Sfrutta un oggetto `Canvas` (la "tela") per disegnare e la classe `Paint` per definire colori e stili. Permette inoltre di gestire gli eventi interattivi (es. il tocco).
- **Classe SurfaceView (Grafica 3D e alte prestazioni):** Basata su un oggetto `Surface`, è ideale per operazioni pesanti come animazioni e giochi. A differenza della `View`, esegue il disegno su un thread in background separato e supporta le librerie OpenGL.

I Metodi Chiave della Classe View: `onMeasure` e `onDraw`

Se si crea una nuova interfaccia 2D estendendo la classe `View`, un'implementazione base fornirà semplicemente un'area vuota. Per rendere il controllo reattivo alle dimensioni dello schermo e capace di mostrare elementi grafici complessi, è obbligatorio sovrascrivere due metodi specifici:

- `onMeasure()` : Calcola e definisce lo spazio (altezza e larghezza) che il controllo deve occupare, rispettando i limiti fisici imposti dal Layout genitore.
- `onDraw()` : È il "motore grafico" del componente, in cui vengono eseguite tutte le istruzioni pratiche per disegnare materialmente forme, colori e testi sul `Canvas` .

I Menù in Android: ActionBar, Toolbar e Menù Contestuali

I menù rappresentano lo strumento principale per consentire all'utente di selezionare e avviare le diverse funzionalità di un'applicazione. Possiamo distinguere due tipologie principali:

- **Menù dell'Activity (Options Menu):** Ogni Activity può definire un proprio menù principale, tradizionalmente attivato tramite la pressione dell'apposito tasto hardware o software del dispositivo.
- **Menù Contestuali (Context Menu):** Sono legati a una specifica *View* e si attivano solitamente tramite una pressione prolungata (*long-press*, per più di 3 secondi) sull'elemento stesso.

Struttura e Usabilità: Il Modello a Tre Livelli

Per ottimizzare l'usabilità sui dispositivi mobili, i menù supportano vari elementi interattivi strutturati su tre livelli gerarchici:

- A. Menù a Icone (Icon Menu):** Menù compatto posizionato nella parte inferiore dello schermo che mostra al massimo **6 elementi**. Utilizza testo e icone (per convenzione in scala di grigi e in rilievo) ma **non supporta** checkbox, radiobutton o scorciatoie da tastiera.
- B. Menù Espanso (Expanded Menu):** Se l'Activity ha più di 6 voci, il sesto elemento diventa automaticamente il pulsante **"More"**. Premendolo, si apre una lista scorrevole per gli elementi extra che supporta testo, checkbox e radiobutton, ma **non le icone**. Premendo "Indietro" si torna al menù a icone.
- C. Sottomenù (Submenu):** Si aprono in una finestra flottante (*floating window*) provvista di intestazione e rappresentano un'alternativa alle strutture ad albero. Supportano testo, checkbox e radiobutton, ma hanno due forti limitazioni: **non mostrano le icone e non supportano gli annidamenti** (non si può inserire un sottomenù dentro un altro). Premendo "Indietro" si torna al livello precedente.

Implementazione nel Codice: Creazione e Gestione

La creazione e la gestione dei menù in un'Activity ruotano attorno a due metodi di *callback* fondamentali.

Creazione: `onCreateOptionsMenu()`

- Viene richiamato automaticamente dal sistema alla prima visualizzazione del menù.
- Le singole voci si inseriscono tramite il metodo `.add()` , specificando quattro parametri fondamentali: **ID di Gruppo**, **ID dell'Elemento** (un identificatore univoco, convenzionalmente una variabile statica inizializzata da `Menu.FIRST`), **l'Ordine** di apparizione e il **Testo** da mostrare.
- Per recuperare il riferimento a un elemento dopo la sua creazione, si utilizza il metodo `findItem(ID)` .

Gestione Click: `onOptionsItemSelected()`

- Centralizza la gestione delle interazioni dell'utente per tutte le voci del menù.
- Riceve in input l'oggetto `MenuItem` selezionato e, generalmente tramite un costrutto `switch` , ne confronta l'ID per eseguire l'azione corrispondente.

Opzioni Avanzate: Checkbox e RadioButton

I menù espansi e i sottomenù supportano opzioni di selezione avanzate:

- **Checkbox (Selezione multipla):** Si attiva su una normale voce di menù invocando `setCheckable(true)` . Il suo stato si legge o modifica tramite il metodo `setChecked()` .
- **RadioButton (Selezione esclusiva):** Permette di selezionare **un solo elemento per volta** all'interno di un gruppo. Si implementa assegnando a tutte le voci interessate lo stesso *ID di Gruppo* e richiamando successivamente il metodo `setGroupCheckable()` , impostando a `true` il parametro *exclusive*.

Personalizzazione Avanzata delle Voci di Menù

Oltre alle impostazioni di base, Android permette di arricchire le singole voci (Item) di un menù con diverse funzionalità:

- **Scorciatoie da tastiera:** Tramite il metodo `setShortcut()` si possono assegnare due tasti rapidi a un elemento, uno per il tastierino numerico e uno per la tastiera completa.

- **Titoli condensati:** La funzione `setTitleCondensed()` imposta un testo abbreviato, ideale nei "menù a icone" dove lo spazio è limitato o per elementi complessi come le checkbox.
- **Icone:** Si assegnano tramite il metodo `setIcon()`. La convenzione raccomanda di utilizzare grafiche in scala di grigi con un leggero effetto incassato o in rilievo.

Gestione delle Azioni e Modifica Dinamica

Oltre al metodo centralizzato standard (`onOptionsItemSelected`), Android offre due alternative per gestire i click sulle singole voci di menu. **Listener Dedicato:** È possibile associare un `MenuItemClickListener` direttamente a una specifica voce tramite il metodo `setOnMenuItemClickListener`. Questa pratica è consentita, ma è generalmente sconsigliata per mantenere il codice pulito e centralizzato. **Intent Diretto:** Si può assegnare un `Intent` a un elemento di menu tramite `setIntent`. Se l'elemento viene selezionato e il click non viene intercettato da un listener, si avvia automaticamente l'attività o il servizio specificato.

Modifica Dinamica a Run-time (`onPrepareOptionsMenu`): È lo strumento ideale per modificare la struttura o lo stato del menu in base al contesto dell'applicazione (ad esempio, per nascondere o disabilitare una voce). Viene invocato dal sistema **immediatamente prima** che il menu venga visualizzato sullo schermo, questo consente di recuperare i riferimenti agli elementi e aggiornarli in tempo reale, garantendo che l'interfaccia rifletta sempre lo stato attuale dell'app.

I Sottomenù (Submenu)

Un sottomenù si attiva quando l'utente seleziona una voce specifica del menù principale, aprendo una lista di opzioni correlate. Anche se visivamente ricordano i menù contestuali, la loro costruzione è diversa. Si utilizza il metodo `addSubMenu()`, che richiede gli stessi parametri del metodo `add()` standard (ID del gruppo, ID dell'elemento, ordine di apparizione e testo). Offre inoltre metodi grafici specifici: `setHeaderIcon()` per mostrare un'icona nell'intestazione della finestra fluttuante del sottomenù, e `setIcon()` per assegnare un'icona alla voce "padre" che lo ha generato.

I Menù Contestuali (Context Menus)

Sono legati strettamente a una specifica `View` e si attivano solitamente tramite una pressione prolungata (*long-press*).

- **Creazione:** Possono essere incapsulati in una `View` personalizzata ridefinendo al suo interno `onCreateContextMenu()`, oppure (più comunemente) gestiti nell'`Activity`, previa registrazione della `View` interessata tramite `registerForContextMenu()`. Per creare e popolare un menù contestuale esistono due approcci principali:
1. **Nell'Activity (Approccio più comune):** Si ridefinisce `onCreateContextMenu()` nell'`Activity`. Affinché il menù si attivi, è **obbligatorio** registrare le `View` interessate chiamando il metodo `registerForContextMenu(view_interessata)`.
 2. **Popolazione Dinamica e Intent Filters:** Il metodo `onCreateContextMenu()` riceve come parametro la `View` che è stata toccata. Questo permette di controllare quale elemento ha scatenato l'evento e popolare il menù dinamicamente con le opzioni più adatte. Inoltre, Android permette di usare gli *Intent Filters* per interrogare il sistema e scoprire se altre app installate offrono azioni compatibili con quella `View`, aggiungendole automaticamente come voci del menù.

Gestione della Selezione: Segue la logica dei menù standard. L'approccio più elegante è ridefinire il metodo `onContextItemSelected()` dell'`Activity`, utilizzando l'ID dell'elemento cliccato per smistare e gestire l'azione richiesta.

Definizione dei Menù tramite file XML (Best Practice)

Android raccomanda di definire i menù in file XML separati dal codice Java/Kotlin, salvandoli nell'apposita directory `res/menu/`. Il nome del file fungerà da identificatore univoco della risorsa (es. `R.menu.nome_file`). Questo approccio permette di sfruttare il sistema di selezione automatica delle risorse di Android, caricando dinamicamente menù alternativi in base all'hardware, all'orientamento dello schermo o alla lingua, senza dover scrivere complessa logica condizionale nel codice.

4 Persistenza Dati e Sicurezza

SQLite

È lo strumento principale per salvare permanentemente dati strutturati. Si tratta di un database **relazionale** (RDBMS) open source, leggero e integrato nativamente nello stack di Android. Tutto ciò minimizza i tempi di latenza, riduce le dipendenze esterne e semplifica la sincronizzazione. I database risiedono nella directory interna dell'app e **sono accessibili esclusivamente dall'applicazione che li ha generati**, garantendo sicurezza e privacy. La versione utilizzata da Android è un'implementazione compatta in linguaggio C.

Condivisione dei Dati: I Content Provider

Mentre SQLite mantiene i dati privati, i **Content Provider** sono lo strumento nativo per esporli e condividerli con altre app. Forniscono un'interfaccia standardizzata che sfrutta il **modello ad URI** (con lo schema `content://`) per indirizzare le risorse.

Garantiscono il **disaccoppiamento** tra il livello applicativo e i dati fisici (nascondendo se i dati arrivano da SQLite o altrove). Qualsiasi app dotata dei giusti permessi può usarli per interrogare o effettuare operazioni (CRUD) sui dati di altre app, comprese quelle di sistema (es. la rubrica).

Esplorazione dei Risultati: L'Oggetto Cursor

In Android, le query eseguite su un database non restituiscono semplici array, ma oggetti **Cursor**, che fungono da veri e propri "puntatori" alla tabella dei risultati generata.

I Cursor forniscono metodi per scorrere agilmente i record (es. `moveToNext()`). Per leggere fisicamente i dati, è necessario prima spostarsi sulla riga desiderata e poi utilizzare metodi di estrazione tipizzati (come `getString()` o `getInt()`) passando in input l'indice numerico della colonna.

Gestione Pratica del Database: SQLiteOpenHelper

È una classe astratta che rappresenta la *best practice* assoluta per gestire in modo automatizzato e sicuro la creazione, l'apertura e l'aggiornamento della struttura di un database.

Si utilizza creando una classe che la estende e sovrascrivendo due metodi fondamentali:

- `onCreate()` : Richiamato solo la prima volta che il database viene creato (qui si inseriscono i comandi SQL `CREATE TABLE`).
- `onUpgrade()` : Invocato quando si incrementa il numero di versione del database (utile per query come `ALTER TABLE` o per le migrazioni di dati).

È possibile creare o aprire un database manualmente usando il metodo diretto `openOrCreateDatabase()`, rinunciando però ai vantaggi della gestione automatica delle versioni.

La Classe SQLiteDatabase e l'esecuzione delle operazioni

È la classe fulcro della gestione dei dati. Offre metodi preimpostati (`insert`, `update`, `delete`, `query`) che incapsulano e semplificano i comandi SQL nativi, riducendo drasticamente il rischio di errori di sintassi e di vulnerabilità informatiche come le *SQL injection*.

Per eseguire direttamente codice SQL nativo (utile per comandi strutturali o che non restituiscono dati), è sempre disponibile il metodo `execSQL()`.

 **Aggiornamento dell'Interfaccia (Refresh):** Ogni volta che si modificano i valori, i Cursor collegati all'interfaccia devono essere aggiornati per mostrare i nuovi dati. Mentre storicamente si utilizzavano metodi manuali (oggi deprecati) come `requery()`, nello sviluppo moderno si usano architetture reattive (*LiveData*, *Flow*) che "ascoltano" automaticamente le modifiche e aggiornano la UI di conseguenza.

Interrogazione del Database (Lettura / query)

Per estrarre i dati senza scrivere query SQL manuali, si utilizza il metodo `db.query()`, che restituisce i risultati incapsulati in un oggetto **Cursor**. Questo metodo accetta una serie di parametri ordinati per strutturare la ricerca:

- `distinct` (per risultati univoci), `table` (tabella), e `columns` (array delle colonne da restituire).
- `selection` e `selectionArgs` (per gestire la clausola WHERE in modo sicuro tramite il placeholder `?`).
- `groupBy`, `having`, `orderBy` e `limit` (per raggruppare, filtrare, ordinare e limitare i risultati).

Modifica dei Dati: insert , update e delete

Le operazioni di scrittura e modifica si avvalgono dell'oggetto **ContentValues**, una struttura a mappa (chiave-valore) in cui la chiave è il nome della colonna e il valore è il dato effettivo.

- **Inserimento (insert):** Si istanzia un `ContentValues`, si inseriscono i dati tramite `.put("colonna", valore)` e si invoca il metodo `db.insert()` passandogli la tabella e l'oggetto creato. Il metodo restituisce l'ID della nuova riga (oppure -1 in caso di errore).
- **Aggiornamento (update):** Funziona in modo simile all'inserimento, ma si invoca `db.update()` passando anche i parametri `selection` e `selectionArgs` per specificare *quali* righe modificare (es. `"id = ?"`). Restituisce il numero di righe

aggiornate.

- **Cancellazione (delete):** Non necessita di `ContentValues`. Si invoca direttamente **`db.delete()`** indicando la tabella, la `selection` (il WHERE) e i `selectionArgs`. Restituisce il numero di righe eliminate. *Attenzione: se si passa null al WHERE, vengono cancellate tutte le righe della tabella.*

Cos'è Firebase e il Realtime Database

È la piattaforma di punta di Google dedicata allo sviluppo rapido di applicazioni Web e Mobile.

- Il cuore pulsante di questa piattaforma è il **Firebase Realtime Database**, un database NoSQL ospitato in *cloud* e accessibile in contemporanea da ecosistemi diversi (Android, iOS, Web). A differenza dei database relazionali tradizionali, i dati vengono archiviati come un unico grande albero in **formato JSON**. Qualsiasi modifica apportata ai dati si riflette istantaneamente e in tempo reale su tutti i dispositivi connessi.

I Vantaggi Principali di Firebase

L'uso di questa architettura offre notevoli benefici strutturali e prestazionali:

- **Sincronizzazione in Tempo Reale:** Le variazioni dei dati vengono mandate all'istante a tutti i client connessi, eliminando la necessità di scrivere codice per richiedere aggiornamenti continui al server.
- **Gestione della Modalità Offline:** Se cade la connessione a Internet, l'app continua a funzionare salvando le modifiche su una cache locale. Al ripristino della rete, Firebase sincronizza tutto in automatico con il cloud.
- **Cross-Platform e Ampia Accessibilità:** Grazie a librerie ufficiali pronte all'uso, si crea il database una sola volta e vi si accede da qualsiasi piattaforma (Android, iOS, framework Web, ecc.).
- **Architettura Serverless:** I dispositivi mobili comunicano direttamente con il database in cloud, eliminando il bisogno di sviluppare o mantenere un server intermedio (middleware).
- **Sicurezza e Controllo Accessi:** Di default il database è bloccato, ma permette di impostare regole di sicurezza granulari per controllare gli accessi. Tutte le comunicazioni sono crittografate (SSL a 2048-bit) e i dati sono sottoposti a backup continui.
- **API REST:** Se una specifica tecnologia non è supportata dalle librerie ufficiali, i dati restano comunque accessibili tramite chiamate API REST standard.

Operazioni di Base: Inserimento e Recupero Dati

L'interazione con Firebase si basa sui **Riferimenti**, ovvero indirizzi specifici (nodi) all'interno del database.

- **Inserimento:** Si crea un oggetto puntato allo specifico URL del nodo desiderato e si utilizza il metodo `setValue()` per inserirvi il dato.
- **Recupero (Lettura):** Essendo un database in tempo reale, non si usano query bloccanti. Si imposta un riferimento al nodo e si aggancia un **evento di callback** (*Listener*), che si attiverà in automatico ogni volta che i dati in quel nodo verranno creati, letti o modificati.

Attenzione: L'Asincronia delle Query

Le interrogazioni al database avvengono in *background*, mentre il resto del codice continua la sua esecuzione senza fermarsi ad aspettare la risposta della rete.

- **Il problema dell'array vuoto:** Se si dichiara una lista, si lancia la query e si tenta di usare la lista nella riga di codice immediatamente successiva, questa risulterà vuota (o nulla). Il codice, infatti, ha eseguito il controllo prima che il download dei dati fosse terminato.
- **La Soluzione:** Qualsiasi logica di manipolazione dei dati (come riempire una lista o aggiornare la grafica) deve risiedere **esclusivamente all'interno del metodo di callback** (es. `onDataChange()`), il quale scatta solo quando i dati sono effettivamente arrivati a destinazione.

La Struttura dei Dati: L'Albero JSON

Firebase Realtime Database abbandona il classico schema relazionale a tabelle e righe tipico di SQL.

- L'intero database è strutturato come **un unico grande albero JSON** formato da nodi interconnessi.
- **Nodi e Chiavi:** Le nuove informazioni inserite diventano nuovi nodi della struttura, ciascuno identificato in modo univoco da una propria **chiave (key)**.
- Tutti i dati vengono di conseguenza archiviati e scambiati sotto forma di classici oggetti JSON.

Le Regole di Sicurezza e Configurazione (Database Rules)

L'accesso ai dati in Firebase non deve mai essere libero; viene gestito tramite un sistema di regole scritte in formato JSON che permettono di definire con precisione i permessi degli utenti. I tre scenari principali sono:

- **A. Modalità Default (Bloccata):** È l'impostazione predefinita. L'accesso in lettura e scrittura è totalmente disabilitato per chiunque (`false`). Le uniche modifiche possibili sono quelle manuali effettuate dall'amministratore tramite la Console di Firebase.
- **B. Modalità Public (Pubblica / Test):** L'accesso in lettura e scrittura è garantito a chiunque (`true`), inclusi gli utenti non registrati. *Attenzione:* Questa configurazione va utilizzata **esclusivamente per i test e lo sviluppo rapido**, per poi passare obbligatoriamente a regole restrittive prima della produzione al fine di evitare manomissioni.
- **C. Modalità User (Utente Autenticato):** È la configurazione standard e sicura per la produzione. In sinergia con **Firestore Authentication**, permette l'accesso in lettura e scrittura solo agli utenti regolarmente loggati. Il sistema garantisce la privacy verificando che l'ID del nodo che si tenta di leggere/scrivere (`$uid`) corrisponda esattamente all'ID dell'utente autenticato (`auth.uid`).

Ottenere un Riferimento al Database

Per eseguire qualsiasi operazione, serve un puntatore alla radice del database. Lo si ottiene tramite la classe `DatabaseReference` utilizzando l'istruzione `FirebaseDatabase.getInstance().getReference()`.

Operazioni di Scrittura di Base (`setValue`)

Salva i dati nel nodo specificato, **sovrascrivendo** tutto il contenuto preesistente.

- Accetta tipi base JSON e **oggetti Java** (che Firebase mappa automaticamente).
- Per non cancellare un intero blocco (es. un profilo utente), è possibile eseguire una *scrittura mirata* puntando il riferimento direttamente al singolo nodo figlio da aggiornare.

Leggere i Dati in Tempo Reale (`addValueEventListener`)

Si aggancia un ascoltatore al riferimento per leggere i dati e le modifiche.

- La callback `onDataChange()` riceve un `DataSnapshot` (un'istanza dei dati) al momento del collegamento e **si riattiva automaticamente a ogni successiva modifica**.
- Tramite l'oggetto `snapshot` si verifica se i dati sono vuoti (`exists()`) e si estraggono i valori (`getValue()`).

Aggiornamento Multiplo e Atomico (`updateChildren`)

Serve a scrivere simultaneamente su nodi figlio specifici in percorsi diversi, senza sovrascrivere l'intero genitore.

- **Scalabilità (Fan-out):** È fondamentale per aggiornare in un colpo solo lo stesso dato salvato in più punti dell'albero JSON.
- **Atomicità:** Garantisce un'operazione sicura; le modifiche simultanee o riescono tutte o falliscono in blocco, evitando di lasciare il database corrotto a metà.

Eliminazione dei Dati

Eliminazione dei Dati in Firebase Per cancellare le informazioni dall'albero JSON, si possono utilizzare due approcci:

- **Metodo diretto:** Invocare la funzione `removeValue()` direttamente sul riferimento del nodo che si vuole eliminare.
- **Assegnare un valore Null:** Passare `null` come parametro a metodi di scrittura come `setValue()` o `updateChildren()`. L'uso di `updateChildren(null)` è particolarmente strategico per eliminare simultaneamente più elementi in percorsi differenti con una sola chiamata API.

Distaccare i Listener (Prevenire Memory Leaks)

Per evitare sprechi di memoria o anomalie quando un'Activity viene chiusa, è fondamentale scollegare i listener non più necessari:

- **Il comando:** Si utilizza il metodo `removeEventListener()` applicato al medesimo riferimento del database a cui l'ascoltatore era stato agganciato.
- **Registrazioni multiple:** Se lo stesso listener è stato aggiunto più volte a un singolo nodo, andrà rimosso un numero corrispondente di volte per essere scollegato del tutto.
- **Gerarchia dei nodi:** Rimuovere un listener da un nodo "padre" **non** scollega in automatico gli ascoltatori presenti sui suoi nodi "figlio"; ogni listener nei sottonodi deve essere rimosso esplicitamente.

Cos'è ARCore

È la piattaforma sviluppata da Google per creare esperienze in **Realtà Aumentata (AR)**. È compatibile con i dispositivi certificati a partire da **Android 7.0 (Nougat)** e offre alcune API anche per iOS, consentendo lo sviluppo di app multiplatforma.

I Tre Pilastri Fondamentali

Si basa su tre funzioni hardware-software:

- **Tracciamento del Movimento:** Permette allo smartphone di calcolare e monitorare costantemente la propria posizione e orientamento nello spazio 3D.
- **Comprensione Ambientale:** Rileva la posizione e le dimensioni delle superfici piane (orizzontali, verticali o angolate, come pavimenti o pareti) e dei feature points.
- **Stima della Luce:** Valuta l'illuminazione dell'ambiente reale per applicare luci e ombre coerenti ai modelli virtuali, aumentandone il realismo.

L'illusione della persistenza spaziale: Grazie alla combinazione dei tre pilastri, ARCore crea l'illusione della persistenza. I modelli 3D si fondono con l'ambiente e rimangono ancorati esattamente dove sono stati posizionati, permettendo all'utente di girarci intorno o di distogliere lo sguardo e ritrovarli nello stesso punto.

Lo Sviluppo in Android (SDK e Rendering)

Per gli sviluppatori, ARCore fornisce **SDK** con API native per accedere a queste funzionalità. L'SDK include app pronte all'uso come `hello_ar_java` e `hello_ar_kotlin`. Queste app di base sfruttano l'interfaccia **OpenGL** per il rendering della grafica 2D e 3D. Il meccanismo tipico consiste nel far rilevare i piani fisici al software, mostrarvi sopra una griglia e permettere all'utente di ancorare un modello 3D con un semplice tocco (*tap*).

Flutter: Cos'è e perché utilizzarlo

È un framework open-source di Google progettato per sviluppare applicazioni multiplatforma (Mobile, Web, Desktop) utilizzando un'unica base di codice (*single codebase*).

I suoi vantaggi principali includono:

- **Hot Reload:** Permette di visualizzare le modifiche al codice quasi in tempo reale senza perdere lo stato corrente dell'app, velocizzando drasticamente lo sviluppo.
- **Prestazioni e UI:** Le app sono compilate in codice nativo (nessuna web-view), garantendo alte prestazioni. L'interfaccia si costruisce tramite una ricca libreria di **Widget** pronti e personalizzabili.
- **Architettura a strati:** È composto dal *Flutter Framework* (per widget e animazioni), dalla *Piattaforma Dart*, dal motore grafico 2D **Skia** (che disegna fisicamente l'interfaccia) e da un *Embedder* (che gestisce la comunicazione con il sistema operativo ospite).

Dart: Il motore dietro a Flutter

È il linguaggio di programmazione, ottimizzato per i client, che alimenta Flutter. Presenta un layout dichiarativo che consente di scrivere l'interfaccia e la logica insieme. Mentre Java resta una scelta solidissima per lo sviluppo esclusivo su Android, la combinazione Dart/Flutter è superiore per ottimizzare i tempi nel multiplatforma.

Caratteristiche Tecniche Fondamentali di Dart

Dart è un linguaggio potente e moderno. Ecco i suoi pilastri tecnici:

- **Tutto è un Oggetto (OOP):** Dart è rigorosamente orientato agli oggetti: ogni singolo elemento deriva dalla classe base `Object`. Oltre ai classici concetti di ereditarietà e interfacce, supporta funzionalità avanzate come i *mixin*.
- **Compilazione Flessibile (Il vero punto di forza):** Offre un'enorme versatilità supportando la compilazione **JIT** (Just-in-Time) per accelerare lo sviluppo con l'Hot Reload, la compilazione **AOT** (Ahead-of-Time) per rilasciare eseguibili nativi performanti, e la traduzione in JavaScript per le applicazioni web.
- **Gestione della Concorrenza (Isolates):** Per il multithreading, Dart non usa i tradizionali thread che condividono la memoria, ma sfrutta gli **Isolates**: entità sicure e indipendenti che comunicano esclusivamente scambiandosi messaggi.

- **Programmazione Asincrona:** Ha un supporto integrato eccellente per le operazioni lunghe (es. chiamate di rete) tramite i costrutti `Futures` e `async/await`.
- **Sicurezza dei Tipi (Type Safe):** Sfrutta controlli statici e a runtime per la sicurezza dei tipi, ma grazie all'inferenza automatica del linguaggio, specificarli nel codice è spesso opzionale.

Gestione degli Errori e Costanti

Gli strumenti di Dart segnalano due categorie di problemi:

- **Warnings (Avvisi):** Segnalano imperfezioni di logica o stile, ma **non bloccano** l'esecuzione dell'app.
- **Errors (Errori):** Sono critici e impediscono la compilazione e l'esecuzione del codice.

Infine, per gestire le variabili che non devono mai cambiare valore durante l'esecuzione dell'app, Dart utilizza le parole chiave `final` e `const`.